

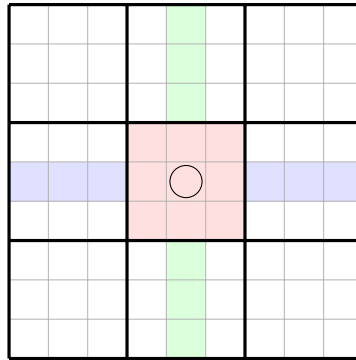
Sudoku

1 The puzzle and its rules

A sudoku grid is a 9×9 board of cells, subdivided into nine 3×3 *boxes*. A puzzle starts with some cells filled in (the *givens*); the task is to fill every remaining cell with a digit from 1 to 9 such that

- every **row** contains each digit exactly once,
- every **column** contains each digit exactly once,
- every **box** contains each digit exactly once.

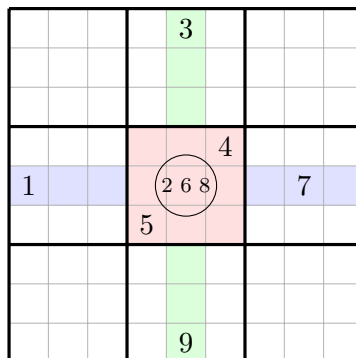
Each cell therefore belongs to three *units* (its row, its column, its box), and its 20 distinct *peers*, the other cells of those units, can never share its digit.



The three units of the circled cell: its row, its column, its box.

2 Candidates and constraint propagation

The solver's central data structure attaches to every empty cell its *candidate set*: the digits not yet ruled out. A digit placed in a cell is erased from the candidates of all 20 peers. In the position below, the circled cell sees 1, 7 in its row, 3, 9 in its column and 4, 5 in its box, leaving candidates $\{2, 6, 8\}$, the small digits a pencil-and-paper solver would write in the corner:



Elimination: $\{1, \dots, 9\}$ minus the row, column, and box gives $\{2, 6, 8\}$.

Eliminations feed two deduction rules, and the rules feed further eliminations:

- **Naked single.** A cell whose candidate set shrinks to one digit must hold that digit. Placing it erases the digit from 20 more candidate sets, which may create new singles.
- **Hidden single.** If, within one unit, some digit remains possible in only one cell, that cell must hold it, even if the cell still has other candidates. Below, the digits 2, 4, 7 are missing from the row; 4 appears in only one candidate set, so it goes there:

1	2 4 7	3	5	2 7	6	2 7	8	9
---	-------	---	---	-----	---	-----	---	---

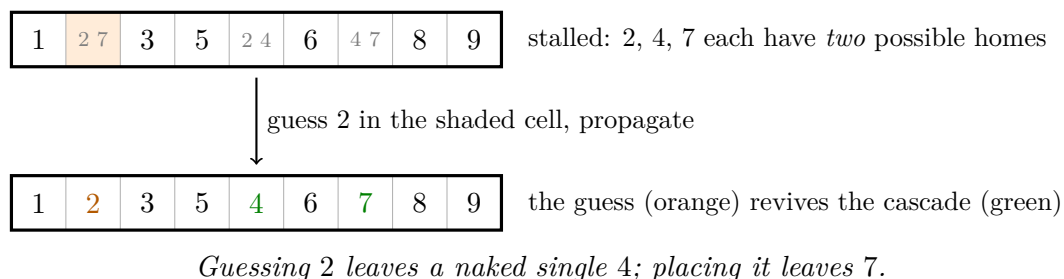
A hidden single: of the three unresolved cells, only the shaded one still has 4 among its candidates.

Running eliminations and both rules until nothing changes is *constraint propagation*: computing a fixpoint. Its power is easy to underestimate. Most published easy and medium puzzles dissolve completely under propagation alone, with never a guess.

3 Search: when logic stalls

The stall. On hard puzzles propagation stalls short of a solution: a full pass over all the rules changes nothing (no elimination, no single), yet some cells still hold several candidates. The fixpoint has been reached, and it is not yet the answer; the rules at hand are exhausted.

The remedy: guess and retract. Pick an unresolved cell, tentatively place one of its candidates, and propagate as if the guess were fact. Two things can happen: either the cascade rolls on, sometimes all the way to a solution, or it hits a contradiction (a cell stripped of every candidate), which refutes the guess; then rewind to the position before it and try the next candidate. Here is the remedy at work on a stalled row:



Guessing well. Recursing on guesses grows the familiar search tree of the earlier quizzes; two refinements keep it tiny:

- **Choose the most constrained cell** (fewest candidates remaining), known as the *minimum remaining values* rule, **MRV** for short. This is the same rescue-the-scarce principle as Warnsdorff's onward counts: a cell with two candidates gives a wrong guess a 50% chance of instant refutation.
- **Propagate after every guess.** Each guess triggers the full cascade, so consequences surface immediately and contradictions appear after a handful of placements, not deep in the tree.

4 Facts worth knowing

- **Uniqueness.** A *proper* puzzle has exactly one solution; verifying properness means searching for a second solution and failing. A puzzle generator must run exactly that check on every candidate puzzle it digs out.
- **Seventeen givens.** No proper 9x9 puzzle with fewer than 17 givens exists (McGuire, Tugemann and Civario, 2012, by exhaustive computer search), and 17-given puzzles do exist.
- **It scales badly, provably.** On generalized $n^2 \times n^2$ boards, sudoku is NP-complete; the 9x9 case is only tamed by its fixed size and by how sharply propagation prunes.

5 From mathematics to program

The program keeps the grid as 81 candidate sets (a solved cell is a singleton) and implements propagation as the fixpoint of the two rules; the solver wraps it in MRV-guided backtracking. Beyond the solved grid, the solver also returns its whole reasoning as a list of typed events, one per action, which the renderer replays:

Given	a starting digit is placed on the board
Naked	a naked single fires (section 2)
Hidden	a hidden single fires in the named unit (section 2)
Guess	propagation stalled; a candidate is tried at the MRV cell (section 3)
Contradiction	a cell lost its last candidate; this branch is dead (section 3)
Backtrack	the search rewinds to the latest guess and tries differently (section 3)

The renderer draws what solvers on paper actually do: pencil marks shrinking as eliminations cascade, deduced digits in one color, guesses in another, and a contradiction flashing before its branch rewinds. Constraint propagation made watchable.

Generating puzzles. The generator runs the machinery backwards. It first builds a random solved grid (the solver on an empty board, choices shuffled), then digs: visiting the cells in random order, it blanks each one and asks whether the puzzle still has exactly one solution, restoring the digit when it does not. The uniqueness question is answered by a solver variant that keeps searching after the first solution and stops at the second. What remains is proper (uniquely solvable) and minimal (every surviving given is load-bearing), typically 22 to 27 givens. Difficulty is graded by how the solver cracks the result: naked singles alone is easy, hidden singles needed is medium, guessing required is hard.